



Institut TIC
de Barcelona

Llenguatges de marques i sistemes de gestió d'informació

JavaScript

1. [Introducció](#)
2. [On va el codi JS?](#)
3. [Resultats del JS](#)
4. [Sintaxi](#)
5. [Events](#)
6. [Strings](#)

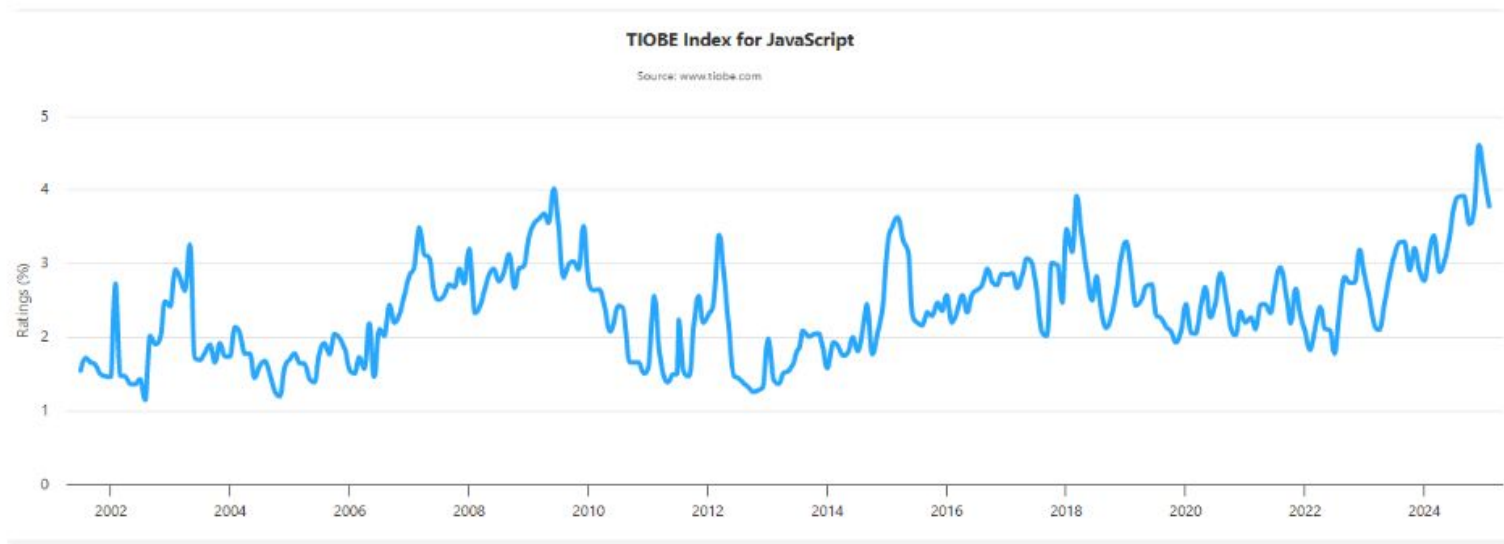
- JavaScript és un dels llenguatges de programació més populars del món

Rank (Feb 2025) Programming Language Ratings Change in Ratings (in 1 year)		
1	Python	23.88% +8.72%
2	C++	11.37% +0.84%
3	Java	10.66% +1.79%
4	C	9.84% -1.14%
5	C#	4.12% -3.41%
6	JavaScript	3.78% +0.61%
7	SQL	2.87% +1.04%
8	Go	2.26% +0.53%
9	Delphi/Object Pascal	2.18% +0.78%
10	Visual Basic	2.04% +0.52%

6. JavaScript

- **Ranking:** 6
- **Learning Curve:** Easy

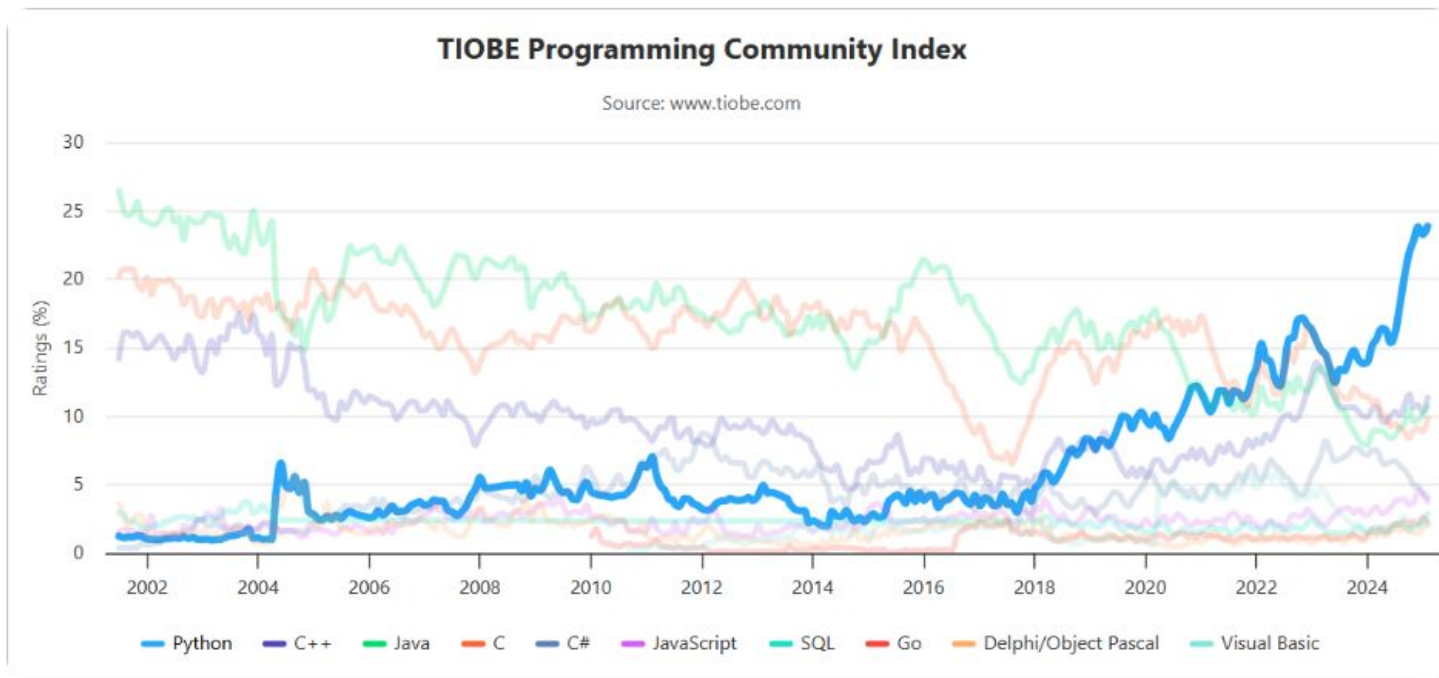
Growth trajectory:



1. Python

- Ranking: 1
- Learning Curve: Easy

Growth trajectory:



- JavaScript és un dels 3 llenguatges que els desenvolupadors web han d'aprendre
 - **HTML** per definir el contingut de les pàgines web
 - **CSS** per definir el format de les pàgines web
 - **JavaScript** per programar el comportament de les pàgines
- Què pot fer:
 - Modificar l'HTML i CSS
 - Calcular, manipular i validar dades
- Com instal·la?
 - No cal descarregar ni instal·lar-ho
 - Ja està correguent en el navegador web o el telèfon

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>El meu primer Javascript</h2>
```

```
<button type="button" onclick="document.getElementById('prova').innerHTML = 'Adeu!'">
```

Marxa

```
</button>
```

```
<p id="prova">Hola!</p>
```

```
</body>
```

```
</html>
```

```
<!DOCTYPE html>
<html>
  <body>
    <h2>Què pot fer Javascript?</h2>
    <p>JavaScript pot canviar l'HTML i el valor dels atributs</p>
    <p>Per exemple podem canviar l'atribut source d'una imatge</p>

    <button onclick="document.getElementById('myImage').src='pic_bulbon.gif'">
      Encén
    </button>

    
    <button onclick="document.getElementById('myImage').src='pic_bulboff.gif'">
      Apaga
    </button>
```

```
<p id="color">Podem canviar l'estil d'un element HTML</p>
<button type="button" onclick="document.getElementById('color').style.fontSize='35px'">
    Canvia
</button>

<p id="amagar">Podem amagar un element HTML</p>
<button type="button" onclick="document.getElementById('amagar').style.display='none' ">
    Amaga
</button>

<button type="button" onclick="document.getElementById('amagar').style.display='block' ">
    Mostra
</button>
</body>
</html>
```

- Generalment el codi JavaScript (JS) es fica entre les etiquetes `<script>` i `</script>`

```
<p id="demo"></p>  
<script>  
    document.getElementById("demo").innerHTML = "Hola món!";  
</script>
```

- Podem afegir codi JS en el body i head del HTML.

```
<!DOCTYPE html>
<html>
<head>
  <script>
    function funcio()
    {
      document.getElementById("demo").innerHTML = "Text canviat";
    }
  </script>
</head>

<body>
  <h2>Demo JavaScript en Head</h2>

  <p id="demo">Un paràgraf</p>
  <button type="button" onclick="funcio()">Prova</button>

</body>
</html>
```

- Ficar els scripts al final del body accelera el render de la pàgina, perquè la interpretació dels scripts triga temps

- Si fiquem el codi JS en un fitxer extern, aquest es pot fer servir des de diferents pàgines web

HTML

```
<h2>JavaScript en fitxer extern</h2>
<p id="demo">Un paràgraf</p>

<button type="button"
onclick="miFuncio()">Prova</button>

<p>Aquest exemple fa servir "miScript.js"
(miFuncio està en "miScript.js")</p>

<script src="miScript.js"></script>
<script src="miScript2.js"></script>
```

miScript.js

```
function miFuncio() {
    document.getElementById("demo").innerHTML =
    "Text canviat.";
}
```

- Ficar el codi en fitxers extern té les següents avantatges:
 - Separa HTML i codi
 - Fa HTML i JS més fàcil de llegir i mantenir
 - Els JS que es queden a la caché pot accelerar la càrrega de pàgines
- Els fitxers de scripts externs no poden fer servir les etiquetes `<script>`
- Els scripts es poden referenciar a fitxers locals o globals (amb una URL)

- JS pot “mostrar” dades de diferents formes:
 - a. Escrivint dins un element HTML amb `innerHTML` o `innerText`
 - b. Escrivint dins HTML amb `document.write()`
 - c. Escrivint a una caixa d’alerta amb `alert()`
 - d. Escrivint a la consola del navegador amb `console.log()`

a) `innerHTML`

- Per accedir al element HTML feu servir `document.getElementById(id)` i aleshores canviar el contingut de la propietat `innerHTML`

```
<p id="demo"></p>
<script>
    document.getElementById("demo").innerHTML = "Hola món!";
</script>
```

b) `document.write()`

Es fa servir **únicament per testejar**. Escriu directament al document

```
<script>
document.write(5 + 6);
</script>
```

Fer servir `document.write()` després de que hagi carregat tot el document, esborrarà tot el contingut HTML.

```
<button type="button" onclick="document.write(5 + 6)">
    Click
</button>
```

c) `alert()`

Mostra una finestra emergent

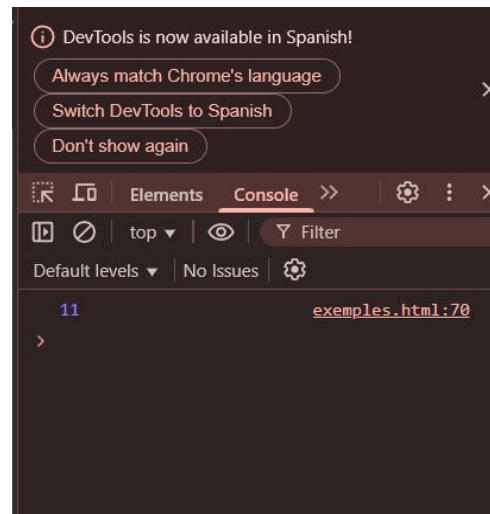
```
<button type="button" onclick="alert('Clandestino');">Atento!</button>
```



d) `console.log()`

Per mostrar dades a la consola

```
<script>  
    console.log(5 + 6);  
</script>
```



- El codi JS està format per declaracions acabades en `;` i els blocs de codi amb `{ }`
- Es poden afegir comentaris amb `//` i `/* */`
- No cal separar-les per línies i els espais en blanc, tabuladors i salts de línia són ignorats

```
a = 5;  
b = 6;  
c = a + b;
```

```
a = 5; b = 6; c = a + b;
```

```
document.getElementById("demo").innerHTML =  
    "Hola Pac!";
```

- **var** i **let**: declara una variable
- **const**: declara una constant
- **if**: marca un bloc de declaracions executat condicionalment
- **switch**: marca un bloc de declaracions a executar per casos
- **for**: marca un bloc de declaracions a executar en bucle
- **function**: declara una funció
- **return**: surt d'una funció
- **try**: implementa tractament d'errades

- **var** es va fer servir a JS des de 1995 a 2015
- **let** i **const** van ser introduïdes a 2015
- **const** es fa servir per valors que no es modificaran
- Declareu les variables amb **let** (o amb **var** si necessiteu donar suport a navegadors antics)

```
const pagues = 14;  
let salari = 2000;  
let salari_anual = salari * pagues;  
  
alert(salari_anual)
```

```
<script>  
let marca = "Volvo" + " " + "Hasselblad";  
document.getElementById("demo").innerHTML = marca;  
</script>
```

- String
- Number
- BigInt
- Boolean
- Undefined
- Null
- Symbol
- Object: pot contenir objects, arrays, dates, maps, sets, intarrays, floatarrays, promises o objectes definits pels usuari

```
// Numeros:
```

```
let longitud = 16;
```

```
let pes = 7.5;
```

```
// Strings:
```

```
let color = "Groc";
```

```
let cognom = 'Parada';
```

```
// Booleans:
```

```
let x = true;
```

```
let y = false;
```

```
// Objecte:
```

```
const persona = {nom:"Ibn",
```

```
cognom:"Battuta"};
```

```
// Array objectes:
```

```
const cotxes = ["Saab", "Volvo", "BMW"];
```

```
// Objecte data:
```

```
const date = new Date("2025-04-27");
```

- La funció es pot executar:
 - Per un event (premer un botó)
 - Invocada des de codi Javascript
 - Automàticament (autoinvocada)

```
function multiplica(p1, p2) {  
    return p1 * p2;  
}
```

- Un event és un esdeveniment que succeeix, per exemple quelcom que el navegador o l'usuari fa. Per exemple:
 - la pàgina web s'ha acabat de carregar
 - s'ha modificat el valor d'un input
 - s'ha premut un botó HTML
- Generalment, volem fer alguna cosa quan succeeix un event, i JS ens permet executar codi quan es detecta un event.
- HTML permet afegir atributs manejadors amb codi JS als elements HTML

```
<element event="codi JavaScript">
```

- Aquest codi JS modifica el contingut de l'element amb id=demo i afegeix la data actual quan es prem el botó

```
<button onclick="document.getElementById('demo').innerHTML = Date()"
  Quina hora és?
</button>
```

o modificar el contingut del propi botó

```
<button onclick="this.innerHTML = Date()"
  Quina hora és?
</button>
```

Com el codi JS pot ser llarg, és comú cridar a funcions

```
<button onclick="mostraData()">Quina hora és?</button>
```

- Alguns events comuns:
 - **onchange**: s'ha modificat un element HTML
 - **onclick**: s'ha premut un element HTML
 - **onmouseover**: l'usuari mou el ratolí a sobre d'un element HTML
 - **onmouseout**: l'usuari mou el ratolí fora d'un element HTML
 - **onkeydown**: l'usuari prem una tecla
 - **onload**: el navegador ha finalitzar de carregar la página

Llista sencera d'events: https://www.w3schools.com/jsref/dom_obj_event.asp

- Coses que podem fer?
 - Coses a fer una vegada s'ha acabat de carregar la pàgina
 - Coses a fer quan es tanca la pàgina
 - Contingut a verificar quan un usuari introdueix informació
 - Accions quan es prem un botó

Llista sencera d'events: https://www.w3schools.com/jsref/dom_obj_event.asp

- Coses que podem fer?
 - Coses a fer una vegada s'ha acabat de carregar la pàgina
 - Coses a fer quan es tanca la pàgina
 - Contingut a verificar quan un usuari introdueix informació
 - Accions quan es prem un botó

Llista sencera d'events: https://www.w3schools.com/jsref/dom_obj_event.asp

- Strings: https://www.w3schools.com/js/js_strings.asp
- Mètodes de strings: https://www.w3schools.com/js/js_string_methods.asp
- Validació de passwd:
https://www.w3schools.com/howto/howto_js_password_validation.asp